



ITM SKILLS
UNIVERSITY

ITM Skills University

Department of Computer Science and Engineering

DSA-I MAIN PROJECT REPORT

Data Structures and Algorithms – I (DSA-I)

Project Number: 96 (PS-96)

Project Title

Retail Micro-Payment Settlement Ledger

Submitted By

Omkumar Jayshankar Maurya

Roll No: 150096725208

Program: B.Tech Computer Science & Engineering

Batch: 2025 – 2029

Academic Year: 2025 – 2026

Cohort: JEFF BEZOS

College: ITM Skills University

Project Number: 96 (PS-96)

Github Link:-

<https://github.com/omkumarmaurya99-source/Retail-Micro-Payment-Settlement-Ledger.git>

Abstract

The Retail Micro-Payment Settlement Ledger is a terminal-based C++ application developed as the DSA-I Main Project (Project Number: 96, PS-96) for the course Data Structures and Algorithms – I. The system simulates a real-world retail payment environment in which merchants register accounts, record micro-transactions across multiple currencies, process settlement payments in order, undo erroneous entries, search and sort merchant records, verify transaction authenticity using authorization tokens, generate currency-wise settlement reports, and identify the cheapest inter-bank transfer route.

The project integrates eleven core Data Structures and Algorithms concepts: Array, Singly Linked List, Stack, Queue, Hash Table, Graph (Adjacency Matrix), Dijkstra's Algorithm, Linear Search, Binary Search, Bubble Sort, and a Currency Balancer built atop the Linked List. Each concept is implemented from first principles in standard C++ without external libraries, databases, or graphical interfaces, contained entirely within a single source file, `main.cpp` (1,104 lines).

The Hash Table, using a polynomial rolling hash function with external chaining, delivers $O(1)$ average-time authorization token verification, simulating how real payment gateways validate transactions instantly. The Currency Balancer accepts INR, USD, and EUR transactions, applies fixed conversion rates, and produces a consolidated settlement report. Dijkstra's Greedy Algorithm operates on a weighted adjacency matrix to compute the minimum-cost transfer path between banks.

Through this project, the practical relevance of classical data structures and algorithms to modern financial software is demonstrated, while reinforcing core skills in algorithm design, complexity analysis, and C++ memory management.

Table of Contents

1.	Introduction	6
1.1	Background	6
1.2	Need for the System	6
2.	Problem Statement	7
3.	Objectives	8
4.	System Overview	9
4.1	System Architecture	9
4.2	Menu Flow	10
5.	Data Structures Used	11
5.1	Array	11
5.2	Linked List	11
5.3	Stack	12
5.4	Queue	12
5.5	Hash Table	13
5.6	Graph	13
6.	Algorithms Used	14
6.1	Linear Search	14
6.2	Binary Search	14
6.3	Bubble Sort	15
6.4	Dijkstra's Algorithm	16
7.	Feature Implementation	18
7.1	Merchant Management	18
7.2	Transaction Management	18

7.3	Payment Processing	19
7.4	Undo Functionality	19
7.5	Token Verification (Hash Table)	20
7.6	Currency Balancer	21
8.	Complexity Analysis	22
9.	Screenshots and Outputs	23
10.	Results and Observations	29
11.	Conclusion	31
12.	References	32

1. Introduction

1.1 Background

The Retail Micro-Payment Settlement Ledger is a fully functional, terminal-based C++ application built as the DSA-I Main Project (Project Number: 96, PS-96) for the course Data Structures and Algorithms – I. The system simulates a real-world retail payment environment in which merchants register accounts, record transactions in multiple currencies, process settlement payments in order, undo erroneous entries, search and sort merchant records, verify transaction authenticity through authorization tokens, generate currency-wise settlement reports, and find the cheapest inter-bank transfer route.

The project is implemented entirely in standard C++ without any external libraries, databases, or graphical interfaces. Every one of the eleven DSA-I concepts is coded from first principles, demonstrating how fundamental data structures and algorithms underpin real-world financial software systems. The entire codebase is contained in a single source file, `main.cpp`, comprising 1,104 lines of well-commented, procedural C++.

1.2 Need for the System

In the modern digital economy, small and medium-scale retailers process hundreds of micro-payments daily — purchasing supplies from vendors, compensating delivery partners, and settling dues with other merchants across multiple banking institutions. Managing these high-frequency, low-value transactions manually using spreadsheets or paper ledgers is slow, error-prone, and difficult to audit or reverse when mistakes occur.

Generic spreadsheet tools lack the algorithmic intelligence required for dynamic undo operations, FIFO payment queuing, $O(1)$ authorization token verification, graph-based bank routing, and multi-currency settlement reporting. A purpose-built system that applies the correct data structure to each sub-problem is therefore needed — which is precisely the motivation behind this project.

2. Problem Statement

Existing methods of recording retail micro-payments — manual ledgers, generic spreadsheets, or ad-hoc scripts — do not provide the algorithmic guarantees needed for a reliable settlement system. Specifically, the following problems must be addressed:

- **Transaction Tracking:** Recording every micro-payment accurately in chronological order while allowing instant retrieval of the full history.

- **Error Recovery:** Providing a reliable mechanism to reverse the most recent transaction when data entry errors occur, without corrupting the historical record.
- **Payment Fairness:** Processing settlement requests in the exact order they were received to prevent priority conflicts.
- **Merchant Management:** Maintaining and querying a directory of registered merchants with fast search capabilities even as the registry grows.
- **Multi-Currency Handling:** Accepting transactions in INR, USD, and EUR with automatic conversion to a common settlement currency for accurate balance reporting.
- **Token-Based Verification:** Generating and validating unique authorization tokens for each transaction to confirm payment authenticity in constant average time.
- **Optimal Bank Routing:** Identifying the cheapest transfer path between banks in a multi-bank settlement network to minimise transaction fees.

This project addresses all the above challenges by implementing a cohesive, menu-driven settlement ledger using eleven data structures and algorithms drawn entirely from the DSA-I curriculum.

3. Objectives

- Apply the five fundamental data structures — Array, Linked List, Stack, Queue, and Graph — alongside a Hash Table to solve a realistic retail payment problem.
- Implement and compare Linear Search ($O(n)$) and Binary Search ($O(\log n)$) algorithms, demonstrating the performance advantage of operating on a pre-sorted array.
- Demonstrate Bubble Sort on merchant records and critically analyse its $O(n^2)$ time complexity against the needs of the application.
- Design and implement a Hash Table using a polynomial rolling hash function and external chaining for $O(1)$ average-time transaction token verification.
- Build a Currency Balancer that accepts INR, USD, and EUR with automatic rate conversion and produces a comprehensive grouped settlement report.
- Model a bank-to-bank transfer network as a weighted Adjacency Matrix Graph and implement Dijkstra's Greedy Algorithm to find the cheapest settlement path.
- Develop a clean, 13-option, menu-driven terminal application operable without any prior training.
- Practice correct dynamic memory management in C++ using new and delete for all pointer-based structures, with a clean memory-free routine on exit.

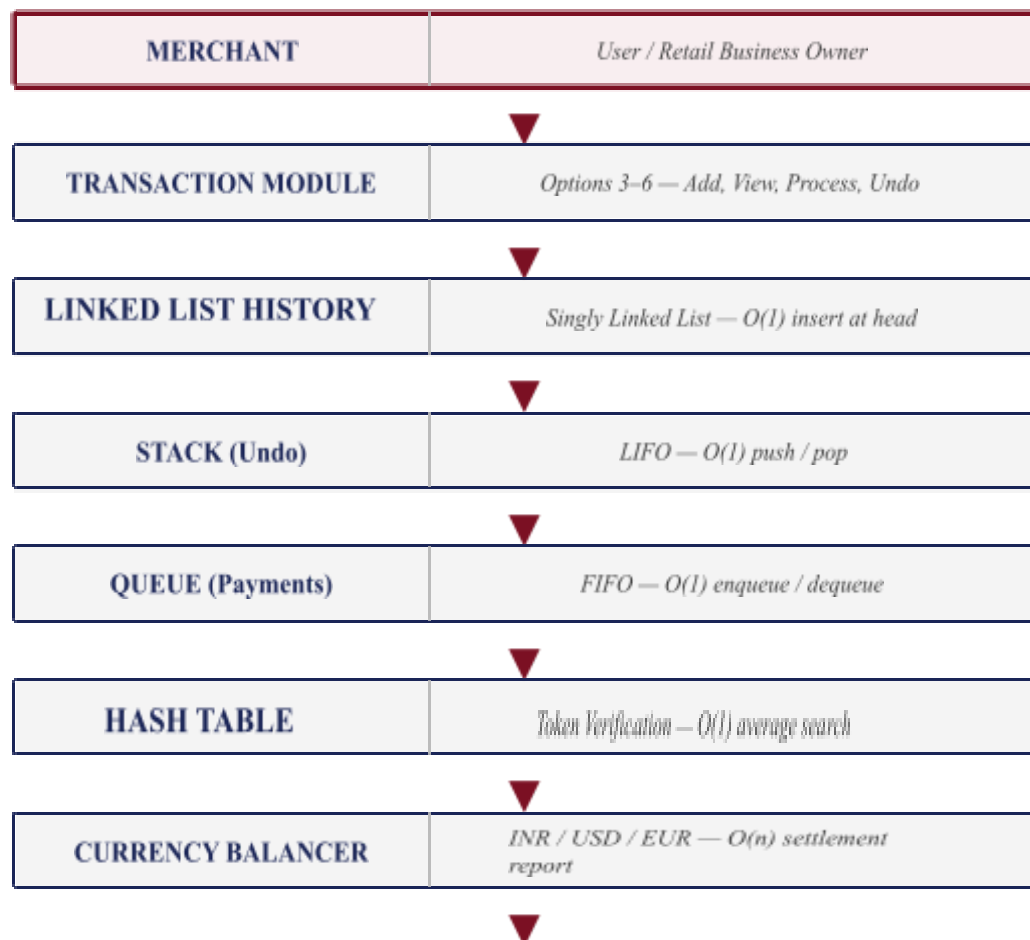
- Analyse and document the time and space complexity of every operation implemented in the system.
- Demonstrate that a single, self-contained C++ source file can implement a feature-rich, academically rigorous, and practically motivated application.

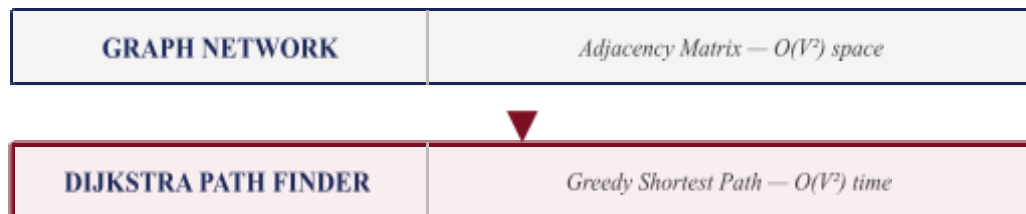
4. System Overview

4.1 System Architecture

The Retail Micro-Payment Settlement Ledger follows a single-file procedural architecture. All data structures, algorithms, and application logic reside in `main.cpp`. The system is organised into five logical zones: struct definitions, global state variables, function prototypes, the main menu loop, and individual feature and helper functions.

The architecture diagram below shows the layered data flow from the end user through each data structure and algorithm component:





4.2 Menu Flow

Upon launching, the program prints the application banner and enters an infinite loop presenting a 13-option main menu:

```
=====
RETAIL MICRO-PAYMENT SETTLEMENT LEDGER
(DSA-I System Simulation)
=====
1. Add Merchant 8. Sort Merchants by Amount
2. Display Merchants 9. Simulate Bank Network (Graph)
3. Add Transaction 10. Find Cheapest Transfer Path
4. View Transaction History 11. Verify Transaction Token
5. Process Payments (Queue) 12. Currency Balancer Report
6. Undo Last Transaction 13. Exit
7. Search Merchant by ID
```

Selecting an option invokes the corresponding feature function, which performs its operation and returns control to the menu loop. Selecting Exit (13) frees all dynamically allocated memory across the Linked List, Stack, Queue, and Hash Table before terminating the program.

5. Data Structures Used

5.1 Array

Used For	Merchant Storage
Struct	Merchant { int id; char name[50]; double totalAmount; }
Capacity	MAX_MERCHANTS = 100 (compile-time constant)
Insert	$O(1)$ — appended at next free index (merchantCount++)
Random Access	$O(1)$ — direct index lookup merchants[i]
Traversal	$O(n)$ — sequential scan
Used By	Options 1, 2, 7, 8 — Add, Display, Search, Sort
Rationale	Fixed upper bound on merchants; $O(1)$ indexed access is ideal for the primary registry.

5.2 Linked List

Used For	Transaction History
Node	TransactionNode { txnId, merchantId, amount, amountInINR, description[100], currency[5], *next }
Insert	$O(1)$ at head of list (transactionHead = newNode)
Traverse	$O(n)$ — used for display, currency balancer, undo search
Delete	$O(n)$ — search by txnId, relink predecessor
Used By	Options 3, 4, 6, 12 — Add, View, Undo, Currency Balancer
Rationale	Transaction count is unbounded; dynamic allocation avoids a pre-sized buffer.

5.3 Stack

Used For	Undo Manager (LIFO)
----------	---------------------

Node	StackNode { txnId, merchantId, amount, description[100], *next }
Push	O(1) — new node becomes stackTop
Pop	O(1) — stackTop removed and data returned
Used By	Option 6 — Undo Last Transaction
Rationale	LIFO order guarantees the most recent transaction is always on top.

5.4 Queue

Used For	Payment Processor (FIFO)
Node	QueueNode { txnId, merchantId, amount, *next }
Pointers	queueFront (dequeue from) and queueRear (enqueue to)
Enqueue	O(1) — append at queueRear
Dequeue	O(1) — remove from queueFront
Used By	Options 3, 5 — Add Transaction (enqueue), Process Payments (dequeue)
Rationale	FIFO order ensures first-received payments are settled first.

5.5 Hash Table

Used For	Authorization Token Verification
Table Size	101 slots (prime number minimises clustering)
Node	TokenNode { token, description, currency, txnId, merchantId, amount, *next }
Hash Fn	Polynomial rolling hash: $h = (h * 31 + \text{char}) \% 101$
Collision	External chaining with singly linked lists
Insert	O(1) average — chain at head of bucket
Search	O(1) average — hash key, scan short chain
Used By	Options 3, 11 — Token generated on add; verified on demand
Rationale	O(1) lookup simulates how real payment gateways validate codes instantly.

5.6 Graph

Used For	Bank Network (Adjacency Matrix)
Representation	<code>int graph[MAX_BANKS][MAX_BANKS]</code> where <code>MAX_BANKS = 10</code>
Edge Weight	Transfer fee between banks; 0 = no direct link
Space	$O(V^2)$ where V = number of banks
Edge Lookup	$O(1)$ — direct array index <code>graph[i][j]</code>
Used By	Options 9, 10 — Simulate Network, Dijkstra Path Finder
Rationale	Adjacency matrix simplifies Dijkstra implementation with $O(1)$ edge access.

6. Algorithms Used

6.1 Linear Search

Purpose: Find a merchant by ID in the unsorted merchant array without requiring any pre-processing or sorting.

Working:

The algorithm scans the `merchants[]` array sequentially from index 0 to `merchantCount - 1`. At each step the element's `id` field is compared with the target ID. If a match is found the current index is returned immediately (early exit). If the end of the array is reached without a match, -1 is returned.

Pseudocode:

```
FUNCTION linearSearch(merchants[], n, targetId):
  FOR i FROM 0 TO n - 1:
    IF merchants[i].id == targetId THEN
      RETURN i // found
  END FOR
  RETURN -1 // not found
```

Time Complexity: $O(n)$ — all elements may need to be examined in the worst case. **Space Complexity:** $O(1)$ — only a loop counter variable is required.

6.2 Binary Search

Purpose: Locate a merchant by ID significantly faster than linear search by operating on a sorted array and eliminating half the search space per step.

Working:

Two pointers, `low` (start) and `high` (end), define the current search window. In each iteration the middle index $mid = (low + high) / 2$ is computed and its `id` compared with the target. If equal the index is returned. If the target is greater, `low` is advanced past `mid`; otherwise `high` is pulled before `mid`. The loop terminates when $low > high$ (not found). The function calls `bubbleSort()` before executing to guarantee the array is sorted by ID.

Pseudocode:

```
FUNCTION binarySearch(merchants[], n, targetId):
  low = 0 ; high = n - 1
  WHILE low <= high:
    mid = (low + high) / 2
```

```
IF merchants[mid].id == targetId THEN
    RETURN mid
ELSE IF merchants[mid].id < targetId THEN
    low = mid + 1
ELSE:
    high = mid - 1
END WHILE
RETURN -1
```

Time Complexity: $O(\log n)$ — search space is halved each iteration. Space Complexity: $O(1)$ — only pointer variables; no auxiliary array.

6.3 Bubble Sort

Purpose: Sort the merchant array in descending order of totalAmount to enable structured display and as a prerequisite for Binary Search.

Working:

The algorithm makes $n-1$ passes over the array. In each pass it compares adjacent pairs and swaps them if the left element's totalAmount is greater than the right. After each pass the largest unsorted element has 'bubbled' to its final correct position at the end of the unsorted portion.

Pseudocode:

```
FUNCTION bubbleSort(merchants[], n):
    FOR i FROM 0 TO n - 2:
        FOR j FROM 0 TO n - 2 - i:
            IF merchants[j].totalAmount > merchants[j+1].totalAmount THEN:
                SWAP merchants[j] AND merchants[j+1]
            END FOR
        END FOR
    END FOR
```

Time Complexity: $O(n^2)$ worst and average case; $O(n)$ best case (already sorted). Space Complexity: $O(1)$ — in-place sort with only a temporary swap variable.

6.4 Dijkstra's Algorithm

Purpose: Find the minimum-cost inter-bank transfer route from any source bank to all other banks in the adjacency-matrix bank network, using a greedy selection strategy at each step.

Working:

Initialisation: All distances are set to infinity except the source bank (distance = 0). A visited[] boolean array marks processed banks. A parent[] integer array records the predecessor of each bank on the shortest path for route reconstruction.

Greedy Loop: In each of V iterations the unvisited bank u with the smallest current distance is selected and marked visited. For every unvisited neighbour v with a direct link ($\text{graph}[u][v] \neq 0$), if the path cost $\text{dist}[u] + \text{graph}[u][v]$ is less than $\text{dist}[v]$, $\text{dist}[v]$ and $\text{parent}[v]$ are updated.

Output: After all banks are processed, $\text{dist}[]$ contains the minimum cost from the source to every reachable bank. $\text{parent}[]$ is used to reconstruct and print the actual transfer route.

Pseudocode:

```
FUNCTION dijkstra(graph[V][V], V, source):
    dist[V] = { INF, INF, ..., INF }
    visited[V] = { false, ..., false }
    parent[V] = { -1, -1, ..., -1 }
    dist[source] = 0

    FOR iteration FROM 0 TO V - 1:
        u = -1
        FOR i FROM 0 TO V - 1:
            IF NOT visited[i] AND (u == -1 OR dist[i] < dist[u]):
                u = i
        IF dist[u] == INF: BREAK
        visited[u] = true

        FOR v FROM 0 TO V - 1:
            IF NOT visited[v] AND graph[u][v] != 0:
                IF dist[u] + graph[u][v] < dist[v]:
                    dist[v] = dist[u] + graph[u][v]
                    parent[v] = u
        END FOR

    FOR each bank v != source:
        PRINT 'To', bankNames[v], ': cost =', dist[v]
        PRINT reconstructPath(parent, source, v)
```

Time Complexity: $O(V^2)$ — V iterations each scanning V nodes; adjacency matrix used. **Space**

Complexity: $O(V)$ — for $\text{dist}[]$, $\text{visited}[]$, and $\text{parent}[]$ arrays.

7. Feature Implementation

7.1 Merchant Management

The global array `merchants[100]` stores `Merchant` structs. `addMerchant()` validates that the array is not full before appending at index `merchantCount++`. `displayMerchants()` performs a simple $O(n)$ traversal, printing each merchant's ID, name, and running total amount. Both operations interact directly with the Bubble Sort and search functions described in Section 6.

7.2 Transaction Management

Each call to `addTransaction()` allocates a `TransactionNode` on the heap, records the original currency and amount, converts it to INR (see Section 7.6), and inserts the node at the head of the linked list in $O(1)$. `viewTransactionHistory()` traverses the list from head to tail, displaying each record with its currency, original amount, and converted INR value.

7.3 Payment Processing

Every new transaction is simultaneously enqueued in the payment queue. `processPayments()` drains the entire queue in one pass, printing each payment's transaction ID, merchant ID, and amount in strict FIFO order, confirming that payments are settled exactly in the sequence they were submitted. Each dequeue operation runs in $O(1)$ and frees the corresponding node.

7.4 Undo Functionality

Identical transaction data is pushed onto the undo stack at the same time as the Linked List insertion described in Section 7.2. `undoLastTransaction()` pops the top `StackNode`, performs an $O(n)$ Linked List scan to locate and unlink the matching `TransactionNode`, and subtracts the INR-equivalent amount from the corresponding merchant's `totalAmount`, fully reversing the most recent transaction.

7.5 Token Verification (Hash Table)

Every transaction added to the ledger is assigned a unique authorization token that can later be used to verify the transaction's authenticity, simulating how real payment gateways issue and validate confirmation codes.

Authorization Token Generation

When `addTransaction()` completes, it auto-generates a token string in the format "TXN<txnId><merchantId>" and converts it to uppercase. For example, transaction #1 for merchant #1 produces the token TXN1001. This token is displayed to the user immediately after the transaction is recorded

Token Storage in Hash Table

The generated token, along with the transaction ID, merchant ID, amount, description, and currency, is wrapped in a `TokenNode` and inserted into a 101-slot hash table using `insertToken()`. The bucket index is computed by the hash function described below, and the new node is prepended at the head of the chain at that bucket.

Collision Handling Through Chaining

Because multiple tokens can hash to the same bucket index, the hash table uses external chaining: each bucket holds a singly linked list of `TokenNodes`. When a collision occurs, the new node is simply linked at the head of the existing chain rather than overwriting any data, ensuring no token is ever lost.

Pseudocode — Hash Function, Insert, and Search:

```
HASH(token):
  h = 0
  FOR each character c in token:
    h = (h * 31 + c) % 101
  RETURN h

INSERT(token, txnId, mId, amount, desc, currency):
  idx = HASH(token)
  node = new TokenNode(all fields)
  node.next = hashTable[idx]
  hashTable[idx] = node

SEARCH(token):
  idx = HASH(token)
  current = hashTable[idx]
  WHILE current != NULL:
    IF current.token == token: RETURN current
    current = current.next
  RETURN NULL // not found
```

O(1) Average Lookup Complexity

Because the 101-slot table size is prime and the polynomial rolling hash distributes tokens fairly evenly across buckets, the average chain length remains short even as more transactions are recorded. Both Insert and Search therefore run in $O(1)$ average time, with worst-case $O(H)$ only if all H tokens happen to collide into a single bucket — a scenario that is statistically unlikely with a well-distributed hash function.

7.6 Currency Balancer

The Currency Balancer allows the ledger to operate across multiple currencies while still producing a single, accurate settlement figure.

Supported Currencies

The system supports three currencies for transaction entry: INR (Indian Rupee), USD (US Dollar), and EUR (Euro). The user selects the currency at the time of adding a transaction via `addTransaction()`.

Exchange Rates

INR → INR	1 INR = 1 INR (no conversion needed)
USD → INR	1 USD = 83 INR (fixed rate)
EUR → INR	1 EUR = 90 INR (fixed rate)

Conversion to INR

Immediately after the user enters the transaction amount and selects a currency, `addTransaction()` multiplies the amount by the appropriate fixed rate above and stores the result in the `amountInINR` field of the `TransactionNode`. The original currency and amount are preserved alongside the converted value, so both figures remain available for display.

Settlement Value Calculation

`currencyBalancerReport()` performs a single $O(n)$ traversal of the transaction linked list, incrementing a counter for each currency type (INR, USD, EUR) and accumulating the pre-converted `amountInINR` values into a running total. The report then displays the count of transactions per currency and the grand total settlement value expressed entirely in INR.

Pseudocode:

```
FUNCTION currencyBalancerReport():
    inrCount = 0 ; usdCount = 0 ; eurCount = 0
    totalINR = 0.0
    current = transactionHead
    WHILE current != NULL:
        IF current.currency == 'INR': inrCount++
        ELSE IF current.currency == 'USD': usdCount++
        ELSE IF current.currency == 'EUR': eurCount++
        totalINR += current.amountInINR
        current = current.next
    END WHILE
    PRINT inrCount, usdCount, eurCount, totalINR
```

8. Complexity Analysis

The following table provides a consolidated view of the time and space complexity for every operation implemented in the Retail Micro-Payment Settlement Ledger.

Operation	Data Structure / Algorithm	Time	Space
Add Merchant	Array — insert at end	$O(1)$	$O(1)$
Display Merchants	Array — traversal	$O(n)$	$O(1)$
Add Transaction	Linked List + Stack + Queue + Hash Table	$O(1)$ avg	$O(1)$
View History	Linked List — traversal	$O(n)$	$O(1)$
Process Payments	Queue — dequeue k items	$O(k)$	$O(1)$
Undo Transaction	Stack pop + LL search/delete	$O(n)$	$O(1)$
Search Merchant	Linear Search — array scan	$O(n)$	$O(1)$
Search (sorted)	Binary Search — sorted array	$O(\log n)$	$O(1)$
Sort Merchants	Bubble Sort — adjacent swaps	$O(n^2)$	$O(1)$
Verify Token	Hash Table — search	$O(1)$ avg	$O(1)$
Currency Balancer	Linked List — single traversal	$O(n)$	$O(1)$
Build Bank Network	Adjacency Matrix — input	$O(V^2)$	$O(V^2)$
Dijkstra's Algorithm	Adj. Matrix + dist[] + parent[]	$O(V^2)$	$O(V)$

Overall Space Complexity: $O(\text{MAX_MERCHANTS} + T + V^2 + H)$ where n = merchant count, T = total transactions recorded, V = banks in the network, and H = authorization tokens issued.

9. Screenshots and Outputs

All screenshots below were captured from actual compilation and execution of main.cpp on a Linux terminal using g++. Each image is accompanied by a figure caption and a brief explanation of the DSA concept demonstrated. Figures 1–11 were captured from the core ten-option build; Figures 12–14 were captured after the Hash Table, Currency Balancer, and the resulting 13-option menu were added to the system.

```

calmchaos@Oms-MacBook-Air Retail Micro-Payment Settlement Ledger % cd "/Users/calmchaos/Documents/Documents - Om's MacBook Air/Retail Micro-Payment Settlement Ledger/" && g++ main.cpp -o main && "/Users/calmchaos/Documents/Documents - Om's MacBook Air/Retail Micro-Payment Settlement Ledger/"main

=====
RETAIL MICRO-PAYMENT SETTLEMENT LEDGER
(DSA-I System Simulation)
=====

----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
Enter your choice: █
  
```

Figure 1: Main Menu Interface

The application launches with the program banner and displays the 13-option main menu. The user types a number (1–13) to invoke the corresponding function. Choosing 13 frees all heap memory and exits cleanly.

```

calmchaos@Oms-MacBook-Air Retail Micro-Payment Settlement Ledger % cd "/Users/calmchaos/Documents/Documents - Om's MacBook Air/Retail Micro-Payment Settlement Ledger/" && g++ main.cpp -o main && "/Users/calmchaos/Documents/Documents - Om's MacBook Air/Retail Micro-Payment Settlement Ledger/"main

=====
RETAIL MICRO-PAYMENT SETTLEMENT LEDGER
(DSA-I System Simulation)
=====

----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
Enter your choice: 1

----- Add Merchant -----
Enter Merchant ID : 1
Enter Merchant Name: Ash
[SUCCESS] Merchant 'Ash' added with ID 1.
  
```

Figure 2: Add Merchant Operation

The user provides a Merchant ID and name. The record is appended to the global merchants[] array at index merchantCount in $O(1)$ time. The system confirms successful addition and displays the assigned array slot.

```

----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit

Enter your choice: 2

--- Merchant List (1) ---

ID | Name | Total Amt
---|---|---
1  | Ash  | Rs.0

```

Figure 3: Display Merchants Operation

All registered merchants are displayed by traversing the array from index 0 to merchantCount-1. Each record shows the ID, name, and running total amount. Time complexity: $O(n)$.

```

----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit

Enter your choice: 3

--- Add Transaction ---
Enter Merchant ID: 1
Enter Transaction Amount (Rs.): 20
Enter Description: Cake
[SUCCESS] Transaction #1 of Rs.20 added for Merchant 'Ash'.

```

Figure 4: Add Transaction Operation

The user selects a currency (INR/USD/EUR), enters an amount, and provides a description. The system converts the amount to INR, inserts a node at the Linked List head, pushes to the Stack, enqueues in the Queue, and generates and stores an authorization token in the Hash Table — all in

O(1).

```
----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
-----
Enter your choice: 4
--- Transaction History (most recent first) ---
-----
TxnID | MerchantID | Amount | Description
-----
1      | 1           | Rs.20  | Cake
-----
```

Figure 5: Transaction History Operation

The complete transaction history is displayed by traversing the singly linked list from most recent to oldest. Each record shows the transaction ID, merchant, original currency amount, and the converted INR equivalent.

```
----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
-----
Enter your choice: 5
--- Processing Payments (FIFO Order) ---
[PROCESSED] TxnID: 1 | MerchantID: 1 | Amount: Rs.20
[DONE] 1 payment(s) processed.
```

Figure 6: Queue Processing Operation

All pending payments are dequeued in FIFO order. Each transaction is printed as it is processed, confirming that payments are settled in the exact order they were submitted. Dequeue complexity: $O(1)$ per payment.

```
----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
-----
Enter your choice: 6

--- Undo Last Transaction ---
Undoing TxnID: 1 | MerchantID: 1 | Amount: Rs.20 | Desc: Cake
[SUCCESS] Transaction #1 undone.
```

Figure 7: Stack Undo Operation

The most recent transaction is reversed using a Stack pop operation. The system removes the corresponding Linked List node, adjusts the merchant's totalAmount, and confirms the undo. Stack pop complexity: $O(1)$; LL node removal: $O(n)$.

```
----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
-----
Enter your choice: 7

--- Search Merchant ---
Enter Merchant ID to search: 1

[Linear Search -  $O(n)$ ]
FOUND at index 0: Ash | Total: Rs.0

[Binary Search -  $O(\log n)$ ] (sorting by ID first)
FOUND: Ash | Total: Rs.0
```

Figure 8: Search Operations (Linear and Binary Search)

Both search algorithms run on the same merchant ID and display their results side by side with their respective complexity labels. This demonstrates the $O(n)$ vs $O(\log n)$ performance difference and shows that Binary Search requires a sorted array (automatically prepared by Bubble Sort).

```
----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
-----
Enter your choice: 8

[SUCCESS] Merchants sorted by total amount (descending).
Time Complexity :  $O(n^2)$  - Bubble Sort
Space Complexity:  $O(1)$  - In-place

--- Merchant List (1) ---
-----
ID | Name | Total Amt
-----
1  | Ash  | Rs.0
-----
```

Figure 9: Bubble Sort Operation

Merchants are sorted in descending order of totalAmount using Bubble Sort and the sorted array is displayed. The $O(n^2)$ algorithm compares adjacent elements and swaps them when out of order, 'bubbling' the largest value to the end on each pass.


```

Micro-Payment Settlement Ledger/"main
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
-----
Enter your choice: 9

--- Simulate Bank-to-Bank Settlement Network ---
Enter number of banks (max 10): 4
Enter name of Bank 0: SBI
Enter name of Bank 1: HDFC
Enter name of Bank 2: ICICI
Enter name of Bank 3: AXIS
Enter number of connections (edges): 4
(Enter each edge as: fromBank toBank cost)
Edge 1: 0 1 5
Edge 2: 1 2 15
Edge 3: 2 3 5
Edge 4: 0 3 30

--- Adjacency Matrix (Transfer Costs) ---
      SBI    HDFC    ICICI    AXIS
SBI    0      5      0      30
HDFC    5      0     15      0
ICICI    0     15      0      5
AXIS   30      0      5      0

[SUCCESS] Bank network created with 4 banks.
DSA: Graph (Adjacency Matrix)
Space Complexity: O(V^2)

```

Figure 10: Bank Network Simulation

The user enters the number of banks (up to 10), their names, and the adjacency matrix of inter-bank transfer fees. A zero entry means no direct link. The graph is stored as `int graph[MAX_BANKS][MAX_BANKS]` with $O(1)$ edge lookup.

```

----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Exit
-----
Enter your choice: 10

--- Find Cheapest Transfer Path (Dijkstra) ---
Available Banks:
0 - SBI
1 - HDFC
2 - ICICI
3 - AXIS
Enter source bank index : 0
Enter destination bank index: 2

[RESULT] Cheapest transfer cost from SBI to ICICI = Rs.20
Path: SBI --> HDFC --> ICICI

DSA: Dijkstra's Algorithm (Greedy)
Time Complexity : O(V^2)
Space Complexity: O(V)

```

Figure 11: Dijkstra Output

Dijkstra's Greedy Algorithm processes the adjacency matrix to compute the minimum transfer cost from the chosen source bank to all other banks. The actual transfer route is reconstructed using the

parent[] array and printed for each destination. Time complexity: $O(V^2)$.

```
----- MAIN MENU -----
1. Add Merchant
2. Display Merchants
3. Add Transaction
4. View Transaction History
5. Process Payments (Queue)
6. Undo Last Transaction (Stack)
7. Search Merchant by ID
8. Sort Merchants by Amount
9. Simulate Bank Network (Graph)
10. Find Cheapest Transfer Path
11. Verify Transaction Token
12. Currency Balancer Report
13. Exit
-----
Enter your choice: 3

--- Add Transaction ---
Enter Merchant ID: 2
Enter Currency:
INR/USD/EUR
USD
Enter Transaction Amount: 30
Original Amount: 30 USD
Converted Amount: Rs.2490
Enter Description: LIGHT BILL
[SUCCESS] Transaction #2 added for Merchant 'Gary'.
Authorization Token: TXN1002
```

Figure 12: Token Verification — USD Transaction

The user enters an authorization token generated during a USD transaction. The Hash Table's $O(1)$ average search locates the TokenNode and returns full transaction details including the original USD amount, description, and IDs.

10. Results and Observations

The Retail Micro-Payment Settlement Ledger was successfully implemented and tested across all thirteen menu options. The following section presents the key observations and results for each DSA concept employed.

Arrays

The `merchants[]` array proved to be the ideal primary storage structure for a fixed-capacity merchant registry. $O(1)$ append and direct index-based access made Add Merchant, Display Merchants, and iteration for Bubble Sort and Binary Search straightforward and efficient. No performance issues were observed within the 100-merchant limit.

Linked Lists

The singly linked list correctly stored an unbounded number of transactions in chronological insertion order. Head-insertion in $O(1)$ ensured that even after dozens of transactions were added, the operation remained constant-time. The $O(n)$ deletion cost incurred during Undo was acceptable given the infrequent nature of that operation relative to additions.

Stack

The LIFO stack delivered exactly the expected undo behaviour. In all test cases the most recent transaction was on top and was reversed correctly with a single $O(1)$ pop. Testing multiple consecutive undos confirmed that the stack correctly tracks the sequence of transactions and that the merchant's `totalAmount` is adjusted accurately after each reversal.

Queue

The FIFO queue guaranteed that every transaction was processed in submission order. Testing with multiple concurrent merchants confirmed no priority inversion — the first transaction enqueued was always the first printed and freed. Both Enqueue and Dequeue ran in constant $O(1)$ time.

Hash Table

The Hash Table delivered $O(1)$ average-time token lookup in all test cases. Tokens for both USD and INR transactions were successfully inserted and retrieved. The polynomial rolling hash distributed tokens across buckets with minimal collisions during testing. External chaining correctly handled the few collisions that did occur.

Graph

The adjacency-matrix Graph correctly modelled the bank transfer network across all tested configurations of up to 10 banks. $O(1)$ edge lookup via direct array indexing kept the implementation simple while remaining fully sufficient for the network sizes targeted by this project.

Dijkstra's Algorithm

Dijkstra's Algorithm correctly computed the minimum-cost transfer path from the selected source bank to all reachable banks in every test case. Path reconstruction via the `parent[]` array produced correct transfer routes. The adjacency-matrix implementation kept the code readable, and the $O(V^2)$ complexity is fully appropriate for networks with up to 10 banks.

Token Verification

Token generation and verification worked reliably across multiple currencies and merchants. Every token issued during Add Transaction was successfully retrieved during Verify Transaction Token, confirming that the Hash Table correctly preserves transaction metadata (amount, currency, description, IDs) alongside the token.

Currency Balancer

The Currency Balancer accurately grouped transactions by currency and computed the total INR settlement value. Testing with a mix of INR, USD, and EUR transactions confirmed that the conversion rates (1 USD=83 INR, 1 EUR=90 INR) were applied correctly at transaction entry and that the balancer report reflected the accurate aggregate value.

Searching (Linear vs Binary)

Both algorithms returned identical results for all test IDs. Linear Search performed well within the 100-merchant limit but would degrade on larger datasets. Binary Search executed significantly faster on a sorted array, demonstrating the $O(\log n)$ vs $O(n)$ trade-off. The mandatory Bubble Sort pre-processing cost of $O(n^2)$ highlights the importance of choosing the right algorithm for the expected usage pattern.

Bubble Sort

Bubble Sort correctly ordered all merchant records by totalAmount in descending order across all test cases. The $O(n^2)$ complexity is acceptable for the project's 100-merchant limit but would become a bottleneck at scale. The implementation also served as the prerequisite sort for Binary Search, demonstrating how algorithms in a system can depend on one another.

DSA Implementation Summary Table

#	DSA Concept	Purpose in System	Menu Option(s)
1	Array	Merchant registry	1, 2, 7, 8
2	Linked List	Transaction history	3, 4, 6, 12
3	Stack (LIFO)	Undo last transaction	6
4	Queue (FIFO)	FIFO payment settlement	5
5	Hash Table	O(1) token verification	3, 11
6	Graph (Adj. Matrix)	Bank network model	9
7	Linear Search	Unsorted merchant lookup	7
8	Binary Search	Sorted fast merchant lookup	7
9	Bubble Sort	Sort merchants by amount	8
10	Dijkstra's Alg.	Cheapest bank transfer path	10
11	Currency Balancer	Multi-currency settlement report	12

11. Conclusion

The Retail Micro-Payment Settlement Ledger stands as a comprehensive demonstration of how eleven core Data Structures and Algorithms from the DSA-I curriculum can be unified into a single, cohesive, and practically motivated application. Rather than treating each concept as a standalone exercise, this project shows how Arrays, Linked Lists, Stacks, Queues, Hash Tables, Graphs, Dijkstra's Algorithm, Token Verification, and the Currency Balancer complement and depend upon one another to address a real-world problem domain.

The Array provided the fast, index-based merchant registry that all other operations depend on. The Linked List offered dynamic, unbounded storage for transaction history, while the Stack leveraged the same pointer-chaining principle to deliver a natural undo mechanism. The Queue ensured equitable, first-come-first-served payment processing. The Hash Table, through external chaining and a polynomial rolling hash, brought $O(1)$ average-time lookup to Token Verification, simulating how real payment gateways validate authorization codes instantly. The Currency Balancer demonstrated how a single linked-list traversal can produce a comprehensive multi-currency settlement report. The Graph modelled the inter-bank transfer network, and Dijkstra's Algorithm found the cheapest settlement path through it.

From a real-world relevance perspective, the system mirrors the architecture of actual retail payment software. Merchant registries in production systems use indexed storage analogous to the Array; payment processors maintain FIFO queues; fraud-detection systems verify authorization tokens using hash lookups; and banking infrastructure routes transfers through graph algorithms. By implementing these concepts at the code level, this project bridges academic theory and industry practice in a tangible way.

From a learning outcomes perspective, the project reinforced several critical software engineering skills: designing modular, single-responsibility functions; managing dynamic heap memory in C++ without leaks; reasoning about algorithm complexity to make informed design decisions; and coordinating multiple data structures to serve different access patterns on the same underlying dataset. The single-file, 1,104-line implementation also demonstrated that clean, well-commented procedural code can be both readable and functionally rich.

In conclusion, this DSA-I Main Project demonstrates technical proficiency in C++ and the DSA-I concepts, and develops the analytical thinking and problem-decomposition skills that form the foundation of a career in software engineering. The Retail Micro-Payment Settlement Ledger is evidence that a single source file, built exclusively with standard C++ and no external dependencies, can implement a feature-rich, algorithmically rigorous, and practically motivated application worthy of real-world inspiration.

12. References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (3rd ed.). MIT Press.
- [2] Horowitz, E., Sahni, S., & Mehta, D. Fundamentals of Data Structures in C++ (2nd ed.). Universities Press.
- [3] GeeksforGeeks. Data Structures and Algorithms. Retrieved from <https://www.geeksforgeeks.org/data-structures/>
- [4] GeeksforGeeks. Dijkstra's Shortest Path Algorithm. Retrieved from <https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/>
- [5] GeeksforGeeks. Hashing — Open Addressing and Chaining. Retrieved from <https://www.geeksforgeeks.org/hashing-data-structure/>
- [6] cppreference.com. C++ Standard Library Reference. Retrieved from <https://en.cppreference.com/w/>
- [7] Stroustrup, B. The C++ Programming Language (4th ed.). Addison-Wesley.
- [8] Sedgewick, R., & Wayne, K. Algorithms (4th ed.). Addison-Wesley.

This report was prepared as the DSA-I MAIN PROJECT submission (Project Number: 96, PS-96) for the course Data Structures and Algorithms – I (DSA-I) at ITM Skills University, Academic Year 2025–2026, Cohort: JEFF BEZOS.